Cyril Georgy
Sylvia Ekström

5 novembre 2014

Origin2013

1 Programme

`Origin2013` est la version restructurée, F90-isée, entièrement modulisée du programme d'évolution stellaire du groupe de Genève. Il est composé d'un programme principal et de 32 modules associés. Il est basé sur la version `origin2010` de `svn/origin2010/trunk/`.

La structure en modules permet de sécuriser les variables : le programme utilisateur ne voit que les variables (ou routines et fonctions) que le module utilisé lui laisse voir. Il faut une double déclaration de ces variables : du côté du module utilisé, il doit en permettre l'accès publiques ; du côté du programme utilisateur, il doit les charger avec le module. L'organisation en module permet également une structure logique des différentes routines du programme. En cas de modification des variables dans un module, il faut recompiler l'ensemble du code, après avoir effacé tous les `*.o` et les `*.mod`.

2 Modules

Certains modules ne contiennent que des variables de base nécessaires au programme et à d'autres modules. D'autres contiennent un ensemble de variables et de routines ou fonctions liées à différents aspects du code d'évolution. Ils sont décrits dans les sous-sections suivantes.

2.1 Modules de variables

Les modules décrits ci-après ne contiennent que des variables utilisées dans les routines.

- `const`: constantes physiques et astrophysiques ;
- `evol`: dimension des tableaux et chemin vers le répertoire `inputs` ;
- `caramodele`: caractéristiques globales du modèle (numéro de modèle, masse, luminosité, pas de temps, perte de masse, etc ...) ;
- `diffadvmod`: diffusion, advection ;
- `equadiffmod`: coefficients de résolution des équations différentielles utilisés dans `henyey` (structure) et `henadv` (advection) ;
- `rotmod`: variables liées à la rotation (Ω, moments cinétiques, circulation méridienne, etc ...) ;
- `strucmod`: variables de la structure (pression, température, densité, gradients, gravité, etc ...).

2.2 Modules avec routines

Les modules décrits ci-après traitent d'un aspect de l'évolution stellaire et de sa modélisation.

2.2.1 `abundmod`

Ce module contient les variables d'abondances, de taux de réactions, de neutrinos, d'énergies. Il contient également deux sous-routines :

- `abundCheck`: qui vérifie l'écart à 1 des abondances ;
- `maxCNO`: qui cherche la couche ayant le maximum de N.

2.2.2 `advection`

Ce module contient les 16 sous-routines du traitement de l'advection :

- `advect`: routine centrale de l'advection, qui appelle toutes les autres et modifie finalement le profil de Ω par l'advection et / ou la diffusion.
- `badv`: détermination des conditions à la surface pour la méthode d'henyey pour l'advection ;

`confi`: détermination de la configuration des zones convectives (rad-conv-rad, conv-rad, conv-rad-conv, entièrement rad, entièrement conv, ...) ;

`coradv`: calcul de la différence entre Ω advecté et l'ancien Ω ;

`ebems`: calcule la composante $\bar{\epsilon}/\epsilon_{\mathrm{m}}$ (énergie gravitationnelle et nucléaire) de la circulation méridienne ;

`echtem`: diverses composantes de la circulation méridienne ;

`gbar`: calcul de la valeur moyenne de g sur une isobare ;

`giadv`: calcul des elements de matrice correspondant aux 4 equations permettant de résoudre la partie advective de l'equation de transport du moment angulaire ;

`gtilgm`: calcul de $\tilde{g}/\bar{g}$: fluctuation de g sur l'isobare par rapport au g moyen de la couche ;

`henadv`: résolution des équations différentielles de l'advection ;

`initia`: initialisation de la circulation méridienne selon une expression simplifiée ;

`momcon`: calcul du moment d'inertie des zones convectives ;

`rechzr`: recherche les zones radiatives et convectives de l'étoile ;

`urs1`: calcul de U_r, la composante radiale de la circulation méridienne ;

`vcirc`: calcul de V_r, la composante horizontale de la circulation méridienne ;

`ziadv`: comme `giadv`, mais pour le centre.

2.2.3 chemicals

Ce module contient les routines qui traitent des modifications des abondances chimiques sous les effets des réactions nucléaires, de la convection et du mélange rotationnel.

`chemold`: homogénéisation des anciennes abondances sur les nouvelles zones convectives avant l'appel à `netnew` ou `netwki` ;

`chemeps`: homogénéisation des réactions nucléaires dans les zones convectives avant l'appel à `netnew` (dans le cas `netwki`, on suit les neutrons et des éléments à courte vie, donc on n'homogénéise pas) ;

`netnew`: calcul des changements des abondances dus aux réactions nucléaires ;

`netwki`: idem, mais dans le cas où on suit le réseau NeNa-MgAl ;

`netalu`: appelés par `netwki`, calcul des éléments de matrices des éléments et des taux de réaction (H-b) ;

`netflu`: idem mais pour l'He-b ;

`chemie`: homogénéisation des abondances dans les zones convectives après `netnew` et `netwki` ;

`mixe`: appelé par `chemeps` pour faire le mélange.

2.2.4 convection

Ce module contient les routines qui traitent de la convection.

`bordn`: recherche les bords des zones convectives ;

`over1`: calcul du bord du coeur modifié par l'overshooting ;

`unders`: calcul de la profondeur de la zone convective externe modifiée par l'undershooting ;

`fconva`: fonction de la convection pour $\ell = \alpha \cdot H_\rho$;

`rechzco`: repere le type de configuration :

- `iconra=0`: entierement convective
- `iconra=1`: entierement radiative
- `iconra=2`: rad—-rad
- `iconra=3`: conv—conv
- `iconra=4`: rad—-conv
- `iconra=5`: conv—rad ;

`konvek`: compteur des zones convectives dans l'enveloppe ;

`CZdraw`: détermination des limites de zones convectives pour impression dans le fichier `.g`.

2.2.5 diffusion

`coedif`: calcul des coefficients de diffusion D_{h}, D_{shear}, D_{circ}, D_{conv} et D_{magn} ;

`couran`: calcul des conditions de Courant pour la diffusion des espèces chimiques ;

`courom`: idem pour le calcul de la diffusion d'Ω ;

`diffbr`: résolution de l'équation de diffusion pour les espèces chimiques ;

`diffom`: résolution de l'équation de diffusion pour Ω.

2.2.6 energy

`energ`: calcul des taux de réaction nucléaire ;

`screen`: calcul du screening ;

`calcrates`: calcul des taux de réaction nucléaire et du screening pour les phases avancées ;

`interpol`: interpolation dans les tables de taux de réaction ;

`netburning`: calcul des modifications d'abondances dues aux réaction des phases avancées ;

`netinit`: initialisation du réseau des phases avancées ;

`contribreac`: définition des réactions ayant lieu dans les phases avancées ;

`readnetZA`: lecture du réseau pris en compte ;

`readreac`: lecture des taux de réactions concernées ;

`inversmat`: inversion de la matrice des changements d'abondances dues aux réactions des phases avancées ;

`nucal`: calcul du flux des neutrinos solaires ;

`nuint`: calcul de l'intégrale pour le flux des neutrinos solaires ;

`enint`: calcul de l'énergie interne du modèle :

- énergie potentielle gravitationnelle ;
- énergie rotationnelle ;
- énergie cinétique du gaz ;
- énergie de la radiation.

2.2.7 envelope

`dreckf`: calcul des α, β, γ du triangle ;

`dreck`: calcul du triangle ;

`ggw`: calcul des P, T et r de l'enveloppe et de l'atmosphère ;

`atmos`: intégration de l'atmosphère ;

`anfitg`: détermination des conditions des trois premières couches de l'enveloppe ;

`diff3`: intégration de l'enveloppe ;

`rsgl`: calcul du membre de droite des équations de structure de l'enveloppe ;

`rsgl1`: idem, mais dans le cas des étoiles massives, avec prise en compte de la pression turbulente ;

`print1`: impression de la structure de l'enveloppe.

2.2.8 EOS

`degen`: calcul de l'équation d'état dans le cas du gaz dégénéré ;

`Compute_sepp`: calcul de variables nécessaires pour l'EOS (appelé dans `dichte`) ;

`dichte`: calcul de l'équation d'état.

2.2.9 geomod

Ce module contient les petites routines utilisées pour interpoler dans la table `ftfp.dat`.

`initgeo`: lecture de la table `ftfp.dat` ;

`geocalc`: interpolation dans la table (application générale) ;

`geomat`: à partir de S_ψ calcul de r_ψ, $S_{\text{isobare}}/(4\,\pi\,r_\psi^2)$, f_T, $g(x_\psi)/\langle g\rangle$ et $\left\langle g^{-1}\right\rangle$;

`geomeang`: à partir de r_ψ calcul de $\left\|g(x_\psi)\right\|\cdot\left(\left\langle g^{-1}\right\rangle/g(x_\psi)\right)$;

`geomEdd`: calcul du Γ_{Edd} maximum possible pour la valeur de $\Omega/\Omega_{\text{crit}}$;

`geom`: à partir de r_ψ calcul de f_P et f_T/f_P

2.2.10 henyey

Ce module contient les routines utilisées pour la résolution des équations de structure selon la méthode de Henyey.

`printheynyey`: écriture du fichier `.v` ;

`Calcvmyhelio`: calcul de l'émission des neutrinos solaires et de μ ;

`gisu`: calcul des éléments des équations $G_{1,2,3,4}$ avec éventuellement décalage du point de calcul (Sugimoto 1970, utilisé pour les phases avancées) ;

`gi`: idem pour le cas normal ;

`zi`: calcul des éléments des équations $Z_{1,2,3,4}$;

`henyey`: résolution des équations de structure selon la méthode de henyey.

2.2.11 inputparam

Ce module contient toutes les variables de la carte des paramètres, ainsi que les routines qui contrôlent les changements de ces paramètres.

`Write_param`: écriture des paramètres d'input s'ils n'ont pas la valeur par défaut ;

`Write_namelist`: écriture de la carte des paramètres ;

`Read_namelist`: lecture de la carte des paramètres ;

`FITM_Change`: calcul du changement de `FITM` lors des traversées du DHR ;

`IMLOSS_Change`: choix de la recette de perte de masse radiative adéquate pour le modèle ;

`INPUTS_Change`: choix des paramètres d'input selon le stade évolutif et la situation du modèle.

2.2.12 interpolation

Ce module contient les petites routines utilisées pour les interpolations linéaires ou quadratiques et les splines :

`indic,fipoi,fipoi1,fipoi2,quint,qua,quad,quad_gg,flin,spline,splint,getd.`

2.2.13 ionisation

Ce module contient les routines qui calculent l'ionisation.

`ionpart`: calcul de l'ionisation partielle de H, He, C, O, Ne, Mg ;

`saha`: résolution de l'équation de Saha.

2.2.14 LayersShift

Ce module contient les routines qui contrôlent la gestion des couches.

`FITMshift`: réajustement des couches après un changement de `FITM` ;

`MdotShift`: réajustement des couches après perte de masse ;

`schrit`: ajout ou suppression de couches selon les différences de diverses variables d'une couche à l'autre (M, L, r, P, Ω, X_{H}, X_{He}, $X_{^{12}\text{C}}$, $X_{^{16}\text{O}}$, $X_{^{20}\text{Ne}}$;

`interx`: sous-routine d'interpolation utilisée pour réajuster les valeurs des variables dans les couches modifiées.

2.2.15 magmod

Ce module contient les variables du champ magnétique, ainsi que la routine qui les calculent.

`Mag_diff`: calcul des variables du champ magnétique et des coefficients de diffusion magnétique $D_{\mathrm{mag,X}}$ et $D_{\mathrm{mag},\Omega}$;

2.2.16 nablas

Ce module contient les routines qui calculent les nablas.

`nabla`: calcul de ∇_{rad} ainsi que ∇_{ad} et ses dérivées ;

`nabgam`: calcul du ∇ dans toute l'étoile selon Eq. (6.39) de Maeder (1997) ;

`grapmui`: calcul de ∇_{μ}

2.2.17 nagmod

Ce module contient les routines nag utilisées par le programme.

`c02agf`: calcul des racines d'une équation polynomiale réelle du n^{e} degré ;

`e02acf`: calcul d'un fit polynomial minimax sur un ensemble de points, par des polynomes de Chebyshev ;

`a02aaf,a02abf,a02acf,c02agt,c02agu,c02agv,c02agw,c02agx,c02agy,c02agz,f06blf`: routines appelées par les deux précédentes ;

2.2.18 omega

Ce module contient les routines qui concernent Ω.

`VcritCalc`: calcul de $V_{\mathrm{crit},1}$, $V_{\mathrm{crit},2}$, $V_{\mathrm{eq}}/V_{\mathrm{crit}}$ et $\Omega/\Omega_{\mathrm{crit}}$;

`dlonew`: calcul de $\mathrm{d}\ln\Omega/\mathrm{d}\ln r$;

`omenew`: calcul approximatif de $\Omega(r)$ sur la base du moment cinétique de l'itération précédente et la nouvelle structure ;

`omescale`: calcul de la correction à apporter au profil de rotation pour assurer la conservation du moment cinétique malgré la perte de masse et répartition sur un certain nombre de couches en surface ;

`OmegaReset`: vérification que la correction du moment cinétique n'a pas entraîné des zones avec $\Omega < 0$. Si c'est le cas, mise à zéro de ces zone et re-calcul de la correction ;

`omconv`: calcul de $\Omega(r)$ dans les zones convectives ;

`momevo`: calcul du moment cinétique ;

`mominertenv`: calcul du moment d'inertie de l'enveloppe ;

`vomcon`: calculde $\Omega(r)$ de départ, avec homogénéisation dans les zones convectives, avant de lui appliquer la conservation locale, diffusion et advection ;

`momspe`: calcul du moment cinétique spécifique ;

`omenex`: calcul approximatif de $\Omega(r)$ en vue du modèle suivant ;

`om2old`: calcul du moment d'inertie (utilisé par `omenex`) ;

`omesta`: calcul de $\Omega(r)$ dans le cas où seule la conservation locale de moment cinétique est prise en compte.

2.2.19 opacity

Ce module contient les routines du calcul de l'opacité.

`opacgn93`: détermination de la ou des table(s) à utiliser ;

`opac,t6interp,readco,opaltab,fity,fitx,interp,smooth`: sous-routines du calcul de l'opacité appelées récursivement par opacgn93 ;

`kappa_out`: extrapolation de l'opacité si les conditions en T et ρ sont en dehors des tables ;

`condTest`: test s'il faut prendre en compte l'opacité conductive ;

`cond`: calcul de l'opacité conductive ;

`kappa`: sous-routine d'entrée dans les routines d'opacité. Appel à `opacgn93` ou `kappa_out` selon qu'on est dans les tables d'opacité ou en dehors.

2.2.20 PrintAll

Ce module contient les routines qui stockent et impriment le modèle complet (intérieur, enveloppe et atmosphère).

`StoreStructure_int,StoreStructure_env,StoreStructure_atm:` stockage des variables des couches ;

`PrintCompleteStructure:` impression de la structure complète ;

2.2.21 SmallFunc

Ce module contient toutes sortes de petites routines et fonctions générales.

`girl:` inversion de matrice ;

`CheckProfile:` Vérification qu'il n'y ait pas un gros saut ponctuel dans une courbe, et élimination si c'est le cas ;

`SmoothProfile:` Smoothing d'une courbe par la technique du *locally weighted scatterplot smoothing* ;

`tridiago:` résolution de l'équation de diffusion ;

`primus,drimus:` fonction $f = \frac{a}{T_9^{e1}} \cdot \exp(-\frac{b}{T_9^{e2}})$ et sa dérivée, utilisée pour le calcul analytique des taux de réactions dans `energ` ;

`secun,dsecun:` idem, fonction $f = \frac{a}{T_9^{e1}} \cdot \exp\left(-\frac{b}{T_9^{e2}} - \left(\frac{T_9}{c}\right)^{e3}\right)$ et sa dérivée ;

`tertiu,dterti:` idem, fonction $f = a + b \cdot T_9^{e1} + c \cdot T_9^{e2} + d \cdot T_9^{e3} + e \cdot T_9^{e4} + f \cdot T_9^{e5}$ et sa dérivée ;

`expf10:` fonction passant du ln au $\log_{10}$: $f = \exp\left(\frac{x}{\log_{10}(e)}\right)$;

`exphi:` fonction $f = 1 - \exp(x)$ avec développement en série limité si $|x| \leq 0.03$: $f = -\left(\left(\frac{x}{3} + 1\right) \cdot \frac{x}{2} + 1\right) \cdot x$;

`pos:` recherche de la position d'une valeur dans une table monotone croissante ou decroissante ;

`ValInterp:` fonction d'interpolation $f = \frac{x_1 \cdot (y_2 - y_3) + x_2 \cdot (y3 - y1)}{y_2 - y_1}$;

`OmInterp:` fonction d'interpolation $f = \frac{x_1 \cdot (y_2 - y_3) \cdot z_1 + x_2 \cdot (y_3 - y_1) \cdot z_2}{(y_2 - y_1) \cdot z_3}$

2.2.22 timestep

Ce module contient les routines qui contrôlent les pas de temps.

`zeit:` calcul du pas temporel suivant ;

`TimestepControle:` adaptation de `xcn` selon la situation du modèle ;

2.2.23 winds

Ce module contient les routines liées à la perte de masse.

`xloss:` calcul de la perte de masse radiative ;

`xldote:` calcul de la perte de moment cinétique ;

`dLmagcalc:` calcul de la perte de moment cinétique due au freinage magnétique ;

`aniso:` calcul de l'anisotropie des vents ;

`corrwind:` correction de la T_{eff} pour l'épaisseur du vent ;

2.2.24 WriteSaveClose

Ce module contient les routines de démarrage et bouclage des modèles.

`write4:` sauvetage du modèle initial au début d'un calcul ;

`read4:` rappel du modèle sauvé dans `write4` ;

`SequenceClosing:` bouclage d'une série de modèles ;

`OpenAll:` ouverture des fichiers d'input/output ;

`CloseAll:` fermeture des fichiers d'input/output ;

3 Carte des paramètres

La carte des paramètres d'inputs pour le programme n'a plus un format fixe comme dans les anciennes versions du code. Elle est remplacée par une série de listes nommées dont certains paramètres peuvent avoir une valeur par défaut. Les paramètres sont regroupés par affinités.

`CharacteristicsParams`

contient le nom de l'étoile et les numéro de modèle. Les paramètres dans cette liste sont :

- `STARNAME`: contient le nom de l'étoile. **NB :** on n'est plus condamné à un nom codé en 9 digits.
- `MODANF`: contient le numéro du fichier `.b` nécessaire au démarrage d'un calcul.
- `NWSEQ`: contient le numéro du premier modèle de la série à calculer.
- `NZMOD`: contient le nombre de modèles calculés dans une série.

Aucun de ces paramètres n'est facultatif.

`PhysicsParams`

contient les choix de la physique à inclure et la phase évolutive. Les paramètres dans cette liste sont :

- `IROT`: permet de calculer des modèles en rotation s'il est posé à `1`.
- `ISOL`: permet de calculer la rotation solide s'il est posé à `1`.
- `IMAGN`: permet de prendre en compte les champs magnétiques s'il est posé à `1`.
- `IALFLU`: permet le calcul des réactions nucléaires du réseau Ne-Na et Mg-Al s'il est posé à `1`.
- `IANISO`: permet de prendre en compte l'anisotropie du vent s'il est posé à `1`.
- `IPOP3`: permet de calculer des modèles à Z=0 s'il est posé à `1`.
- `IBASNET`: permet l'utilisation du réseau nucléaire étendu (implémentation future, ne pas mettre à `1` pour le moment).
- `PHASE`: indique les stades de fusion (remplace les anciens paramètres `ipout` et `inetburn`) :
  - `PHASE=1` pour H-b ;
  - `PHASE=2` pour He-b ;
  - `PHASE=3` pour C-b ;
  - `PHASE=4` pour Ne-b ;
  - `PHASE=5` pour O-b ;
  - `PHASE=6` pour Si-b.

`imagn`, `ianiso`, `ipop3` et `ibasnet` sont facultatifs et par défaut posés à `0`.

`CompositionParams`

contient les caractéristiques chimiques du modèle. Les paramètres dans cette liste sont :

- `ZINIT`: mémorise la métallicité intiale de l'étoile (nécessaire dans le calcul des vents WR).
- `ZSOL`: définit la métallicité solaire considérée (nécessaire dans la mise à l'échelle de la perte de masse).
- `Z`: abondance des éléments non pris en compte.
- `IKAPPA`: choix du type d'opacités :
  - `1` anciennes opacités tabulées (Anders & Grevesse 1991 + Kurucz + H§bner) ;
  - `5` opacités tabulées d'OPAL (valeur recommandée) ;
  - `9` opacité par la diffusion des électrons libres.
- `IOPAC`: ce paramètre permet de choisir la table d'opacité désirée dans le cas `ikappa=5` :
  - `1` Grevesse & Noels 1993 (mélange solaire) ;
  - `2` Grevesse & Noels 1993 (mélange alpha-enhanced) ;
  - `3` Asplund et al. 2005 + Ne selon Cunha et al. 2006 (mélange solaire) ;
  - `4` Asplund et al. 2005 + Ne selon Cunha et al. 2006 (mélange alpha-enhanced).

ATTENTION : `zsol` doit être conforme aux tables utilisées. Pour 1 et 2, `zsol=0.020` ; pour 3 et 4, `zsol=0.014`.

`imagn`, `ianiso`, `ipop3` et `ibasnet` sont facultatifs et par défaut posés à `0`.

`RotationParams`

contient les options pour le traitement de la rotation. Les paramètres dans cette liste sont :

`IDIFF:` permet de calculer la diffusion dans les modèles en rotation s'il est posé à `1`.

`IADVEC:` permet de calculer l'advection dans les modèles en rotation s'il est posé à `1` (seulement pendant la séquence principale).

`ISTATI:` permet de calculer les modèles avec la seule conservation du moment cinétique s'il est posé à `1`.

`ICOEFF:` permet de choisir la prescription de rotation désirée selon le tableau suivant :

D_{shear}	D_{h}		
	Zahn 1992	Maeder 2003	Mathis & Zahn 2004
Maeder 1997	11	12	13
Talon & Zahn 1997	21	22	23

`FENERG` fraction de l'énergie du cisaillement utilisé pour le mélange ;

`RICHAC:` nombre de Richardson critique ;

`IGAMMA:` traitement du shear selon Maeder(1997) A&A **321** 134 s'il est posé à `1`.

`FREIN:` permet le calcul du freinage magnétique s'il est $\neq$ `0`. La valeur entrée est celle du champ magnétique appliqué, en Gauss.

`RAPCRILIM:` permet de définir le rapport Ω_{crit} maximal accepté. Si la surface de l'étoile dépasse cette valeur, la perte de masse équatoriale mécanique entre en jeu. S'il n'est pas posé `=0.00000`, il entraîne le traitement de la conservation du moment cinétique.

`VWANT:` vitesse désirée sur la ZAMS. On peut entrer sa valeur de diverses manières :

- υ_{eq} si la valeur est > 1.0
- $\upsilon/\upsilon_{\text{crit}}$ si la valeur est $0.0 <$ `vwant` < 1.0
- $\Omega/\Omega_{\text{crit}}$ si la valeur est $-1.0 <$ `vwant` < 0.0

`XFOM:` facteur multiplicatif du Ω en surface. Permet d'accélérer l'étoile pendant la phase de convergence vers la ZAMS ;

`OMEGA:` Ω en surface (change automatiquement) ;

`XDIAL,IDIALO,IDIALU:` activent le lissage des profils de Ω et μ (lors du calcul de l'advection) ;

Sont facultatifs `istati=0`, `fenerg=1.0`, `richac=1.0`, `gamma=0`, `frein=0.0`, `vwant=0.0` et `xfom=1.0`.

`SurfaceParams`

contient les paramètres de surface (perte de masse, fitm, triangle). Les paramètres dans cette liste sont :

`IMLOSS:` choix du type de perte de masse :

`0` pas de perte de masse ;

`1` prescription de de Jager (1988) ;

`2` perte de masse manuelle (gérée par `fmlos`) ;

`3` formule de Reimers (1975,1977, η donné par `fmlos`) ;

`4` perte de masse WR :

- WNL : Nugis et al. (1998) ;
- WNE et WC : Langer (1989) modifé par Schmutz (1997) ;

`5` prescription de Kudritzki & Puls (2000) pour $M > 15M_{\odot}$ et $T_{\text{eff}} \geq 3.95$, sinon de Jager (1988) ;

`6` prescription de Vink et al. (2001) pour $M > 15M_{\odot}$ et $T_{\text{eff}} \geq 3.95$, sinon de Jager (1988) ;

`7` perte de masse WR selon Nugis & Lamers (2000), avec dépendance en Z selon Eldridge & Vink (2006) ;

`8` perte de masse WR selon Gräfener & Hammann (2008) ;

9 prescription de Kudritzki (2002) pour $T_{\rm eff} > 4.6$ et très bas Z ;

10 prescription de van Loon et al. (2005) pour RSG et AGB ;

11 prescription de Vink et al. (2001) modifiée par Markova & Puls (2008, priv. comm. Puls 2010) ;

`FMLOS`: sauf pour `imloss=2,3`, facteur de multiplication de la perte de masse. Posé à `=0.85` pendant la séquence principale, il est passé `=1` après, et `=3` dans le cas de couches supra-Eddington ;

`IFITM`: permet de gérer les changements de `fitm` lors du passage vers le rouge :

0 le changement doit se faire à la main ;

1 le changement est automatique et vérifie que `fitm` se trouve toujours dans la zone convective externe ;

2 le changement est automatique et utilise un fit polynomial pour modifier `fitm` (pas recommandé dans le cas `irot=1`) ;

3 le changement se fait "comme à la main", à chaque pas de temps ;

4 le changement se fait "comme à la main", à chaque fin de série ;

5 le changement se fait "comme à la main", seulement en montée (facilitation du retour vers le bleu) ;

`FITM`: masse incluse dans l'intérieur de l'étoile ;

`DELTAL,DELTAT`: largeur du triangle en L et T en surface ;

`NNDR`: règle le comportement de $(L,T_{\rm eff})$ par rapport au triangle :

0 pas de vérification que les L et $T_{\rm eff}$ du modèle convergé sont bien dans le triangle ;

1 vérification et recherche d'un autre triangle si ce n'est pas le cas ;

2 une tolérance de 20% sur le triangle est appliquée lors de la vérification ;

Le seul paramètre facultatif est `nndr=1`.

`ConvectionParams`

contient les paramètres du traitement de la convection. Les paramètres dans cette liste sont :

`ILEDOU`: permet de prendre le critère de Ledoux pour la convection s'il est posé à `1`.

`IDIFCON`: permet de calculer la convection comme une diffusion s'il est posé à `1` (utilisé pendant O-b et Si-b).

`ELPH`: longueur de mélange pour la convection de surface. Il est d'habitude posé à 1.6 pour les étoiles avec $M \leq 40 M_\odot$ et à 1.0 pour les plus massives.

`MY`: permet d'intégrer l'enveloppe sur la densité plutôt que la pression s'il est posé à `1` (pour $M > 40 M_\odot$).

`IOVER`: permet de calculer l'overshooting s'il est posé à `1`.

`DOVHP`: valeur du paramètre α d'overshooting $\Lambda = \alpha H_P$.

`IUNDER`: permet de calculer l'undershooting s'il est posé à `1`.

`DUNDER`: valeur du paramètre d'undershooting.

Sont facultatifs `iledou=0`, `idifcon=0`, `iover=1`, `dover=0.30`, `iunder=0` et `dunder=0.0`.

`ConvergenceParams`

contient les paramètres modifiant la convergence du modèle. Les paramètres dans cette liste sont :

`GKORM`: marge d'erreur acceptée sur les variables de structures ;

`ALPH`: fraction de la correction appliquée pour l'itération suivante ;

`AGDRSPT`: valeur absolue de la correction maximale tolérée sur r, `s`, P et T ;

`FAKTOR`: paramètre permettant de "compenser" (pour le calcul) des couches avec $L_r > L_{\rm tot}$;

`DGRP,L,Y,C,O,20`: variation relative (d'une couche à l'autre) tolérée sur P, L, $X(^4{\rm He})$, $X(^{12}{\rm C})$ et $X(^{16}{\rm O})$;

`NBCHX`: nombre d'itérations du calcul de changement de composition chimique ;

`NRBAND`: nombre d'itérations de la chimie entre le modèle `n` et `n+1`.

Sont facultatifs `dgro=0.010`, `dgr20=0.010`, `nbchx=200` et `nrband=1`.

`TimeControle`

contient les paramètres modifiant le pas de temps. Les paramètres dans cette liste sont :

`XCN`: facteur de multiplication du pas temporel pour le calcul d'une série.

`ISLOW`: permet la version `slow`, `extra-slow` ou `mega-slow` du programme s'il est posé à `1`, `2`, ou `3` (utilisé parfois lors des phases avancées).

`ICNCST`: permet de bloquer `xcn=1.0`;

Le seul paramètre facultatif est `icncst=0`.

`VariousSettings`

contient divers paramètres touchant au calcul. Les paramètres dans cette liste sont :

`PLOT`: permet la génération des fichiers nécessaire pour l'affichage de l'évolution en direct avec `pgplot` si posé à `T`.

`REFRESH`: permet le rafraîchissement des fenêtre `pgplot` si posé à `T`.

`IAUTO`: permet de gérer les changements automatiques des paramètres à travers les différentes phases de l'évolution :

- `0` les changements doivent se faire à la main ;
- `1` seuls les changements "évidents" sont faits automatiquement ;
- `2` le programme gère également automatiquement les modifications de `gkorm`, `alph`, `faktor`, `deltal`, `deltat` dans les cas où l'étoile peine à converger.

Dans tous les cas où `iauto`$\neq$`0`, les changements faits automatiquement par le programme sont consignés dans le fichier `input_changes.log` dans le répertoire de calcul.

`IPRN`: définit tous les combien de modèles on imprime la structure complète pendant le calcul d'une série. Si `iprn` $>$ `nzmod`, une seule structure sera imprimée, celle correspondant au modèle `nwseq`.

`IA`: appel ou non à `dreckf` avant le dernier appel à `henyey`.

`IOUT`: nombre de couches utilisées pour adoucir les gradient de diffusion au bord des zones convectives.

`ITMIN`: nombre minimal d'itérations dans `henyey` ;

`XYFILES`: création des fichiers `.x` et `.y` pour post-processing du processus-*s* si posé à `TRUE`.

`IDEBUG`: rajout d'impressions pour débuggage, différents niveaux possible. Toute valeur $>$ `0` met le programme en mode `verbose`.

Sont facultatifs `iprn=99`, `ia=1`, `iout=0`, `itmin=5`, `xyfiles=false` et `idebug=0`.

4 Routines

4.1 Programme principal

Le programme d'appel est `mainrgva2009.f90`. En très résumé, sa structure est la suivante :

- lecture de la structure de départ ;
- calcul de la déformation due à la rotation ;
- calcul de la perte de masse (appel à `xloss, xldote, aniso`);
- intégration de l'atmosphère et de l'enveloppe (appel à `dreck` et sous-routines subséquentes) ;
- lissage des profils de Ω et μ (appel à `grapmui, dlonew`);
- résolution des équations de structure interne (appel à `henyey`) ;
- vérification de l'acceptabilité de la solution. En cas de non-convergence, on repart avec un pas de temps 2x plus petit ;
- calcul des caractéristiques du modèle convergé ;
- réintégration de l'enveloppe (appel à `dreck`) ;
- dernière itération des équations de structure (appel à `henyey`) ;
- calcul de la composition chimique et du profil de rotation du modèle approximatif suivant (appel à `chemold, netnew, omenex`);
- impression du modèle convergé et sauvegarde du modèle approximatif pour calcul d'une nouvelle série.

4.2 Résolution des équations de structure interne

La routine est `henyey` dans le module `henyey_solver.f90`. En très résumé, sa structure est la suivante :

- première partie, traitement couche par couche :
  - résolution de l'équation d'état (appel à `dichte`) ;
  - calcul des opacités, des gradients adiabatique et radiatif et des taux de réactions nucléaires (appel à `kappa, nabla, energ`) ;
  - résolution des équations de structure selon la méthode d'Henyey et al. (1964), par inversion de matrices (appel à `gi, zi`) ;
- deuxième partie, traitement de l'ensemble des couches :
  - calcul de la variation de Ω et des abondances chimiques (appel à `vomcon, omenew, netnew`) ;
  - calcul de la diffusion-advection (appel à `diffom, advec`) ;
- itération jusqu'à convergence.

4.3 Traitement de la rotation

Le traitement de la rotation est réparti sur diverses sous-routines :

- calcul de la déformation de la surface par les appels à `geocrit, geom, geomat, geomeang` ;
- calcul du facteur multiplicatif de la perte de masse (`alpro6`) dans `main` ;
- calcul de l'anisotropie des vents dans `aniso` ;
- calcul de la perte de moment cinétique total sur le pas de temps dans `xldote`, et calcul éventuel de la perte de masse mécanique si l'étoile part à la rupture ;
- calcul des coefficients de diffusion dans `coedif` ;
- calcul des nouvelles abondances après diffusion dans `diffbr` et du moment cinétique dans `diffom` ;
- calcul du terme d'advection dans `advec` et sous-routines subséquentes ;

5 Utilisation

5.1 Récupération du programme

Le programme peut être récupéré du dépôt SVN grâce à la commande :

```
svn checkout --username #username_mail
     https://subversion.unige.ch/users/ekstrom/Origin2013/tags/Origin2013_2.1
```

Le répertoire qui va s'installer contient :

- tous les .f90
- le fichier `make_evosolL`
- le répertoire `inputs` contenant
  - les fichiers `opaAlph_AspCun06.dat`,`opaAlph_GN93.dat`,`opaSol_AspCun06.dat`,`opaSol_GN93.dat` et `kappa93.dat` contenant les opacités
  - le fichier `ftfp.dat` contenant les valeurs de différentes variables utilisées dans le calcul de la déformation due à la rotation
  - les fichiers `vit.datCNE`,`vit.datCNEO` contenant les réactions nucléaires des phases avancées
  - les fichiers `netinit.inCNE, netinit.inCNEO` décrivant le réseau avancé
  - le fichier `liste_tables3` contenant les chemins vers les fichiers de taux de réactions nucléaires pour les phases de combustion de l'hydrogène et de l'hélium
- le répertoire `taux` contenant
  - tous les fichiers de taux de réactions nucléaires listés dans `inputs/liste_tables3`

ATTENTION : Il faut modifier les chemins d'accès dans `evol.f90` et `inputs/liste_tables3`.

5.2 Compilation

Plusieurs compilateurs sont proposés dans `make_evosolL`. Ils permettent de pouvoir traquer les erreurs de codage, mais pour les calculs on recommande d'utiliser `gfortran`.

Pour ne pas envahir le répertoire des routines avec les `⋆.o` et `⋆.mod` résultant de la compilation, on recommande de créer un répertoire `make`, dans lequel on déplace le fichier `make_evosolL` et on procède à la compilation en tapant la commande `make -f make_evosolL`. Si cette structure de répertoire est retenue, il faut s'assurer que la variable `SRCDIR` dans le fichier `make_evosolL` est bien posée à "..".

6 Calcul d'un modèle

6.1 Généralités

Le programme d'évolution stellaire se lance dans le répertoire de calcul avec la commande :

```
#chemin/#programme < #fichier_initial
```

avec `#fichier_initial` pouvant être un fichier `ini⋆` (voir ci-dessous, section **??**) ou le `P⋆.input`.

NB : afin de pouvoir utiliser la version avec visualisation (`PLOT = T`), il faut s'assurer que le fichier `.bash_profile` à la racine contienne bien les variables globales nécessaires au fonctionnement de pgplot. Les lignes suivantes doivent y être, avec bien entendu les chemins adaptés à l'architecture propre de l'utilisateur pour pointer vers les librairies pgplot :

```
export PATH=/opt/local/bin:/opt/local/sbin:$PATH
PGPLOT_DIR="/opt/local/lib"
PGPLOT_FONT="/opt/local/share/pgplot/grfont.dat"
PGPLOT_RGB="/opt/local/share/pgplot/rgb.txt"
PGPLOT_DEV="/XSERVE"
export PGPLOT_DIR
export PGPLOT_FONT
export PGPLOT_RGB
export PGPLOT_DEV
```

6.2 Création des fichiers initiaux

Dans le répertoire de calcul, lancer le programme `inifile` (voir section **??**). Le programme demande la masse et la métallicité du modèle, puis s'il est en rotation (1) ou non (0), puis si le type de structure et de fichiers par défaut convient (format `Origin2013`, abondances solaires d'Asplund et al. 2005 avec Ne de Cunha et al. 2006).

Le programme génère un fichier `ini⋆`, ainsi que les fichier `netdef.in` et `netalu.dat`.

6.3 Pré-ZAMS

Dans le cas **sans rotation**, les étapes sont les suivantes :

- calcul d'une série de 10 modèles depuis le ini⋆.com ;
- calcul d'encore 2 séries de 10 à la main depuis le P⋆.input ;
- poser `GKORM= 0.10` et lancer avec `Origin_launch.py`.

Dans le cas **avec rotation**, on doit converger vers la vitesse voulue (`vwant`) sur la ZAMS ($X_c = X_{ini} - 0.003$) en accélérant progressivement l'étoile et en laissant assez de temps pour que la structure se relaxe. Le programme le fait automatiquement. Les étapes à suivre sont les suivantes :

- vérifier la valeur de `vwant` ;
- calcul d'une série de 10 modèles depuis le ini⋆.com ;
- lancer avec `Origin_launch.py` ;
- le programme s'arrête automatiquement sur le modèle de ZAMS ;
- on le lance une fois tel quel à la main ;

- on pose `NZMOD= 10` et `GKORM= 0.10` et on peut lancer le calcul avec `Origin_launch.py`.

NB : dans le cas où `ISOL=1` dans cette phase, l'accélération se fera à chaque pas de temps. Pour une accélération plus propressive, poser `ISOL=2`.

6.4 Séquence principale

Le programme doit tourner sans accroc durant cette phase.

6.5 Fusion de l'hélium

Si `IAUTO=1`, le programme change automatiquement les paramètres comme suit :

- `GKORM= 0.30, FAKTOR= 1.00+01;`
- `FMLOS= 1.000+00` dès que l'étoile par vers le rouge.

Dans le cas **avec rotation**, seule la séquence principale est calculée avec le terme d'advection, donc le programme change également :

- `IADVEC= 0;`
- `XDIAL= 0.000, IDIALO= 0, IDIALU= 0.`

Souvent la phase de fusion de l'hélium est délicate à calculer (surtout pour les très massives). Le programme a parfois de la peine à converger en surface. Dans ce cas, on peut poser `IAUTO= 2` et `NNDR= 2`, ce qui donne plus de marge sur le triangle et le modifie si nécessaire. On peut sinon changer le triangle à la main chaque fois que ça coince... Si le nombre de couches est suffisant, on peut aussi légèrement augmenter `DGRP`.

6.6 Phases avancées

Si `IAUTO=1`, le programme change automatiquement les paramètres comme suit à la fin de l'hélium :

- `IOVER= 0;`
- `AGRDSPT= 0.10D-05;`
- `FAKTOR= FAKTOR+10.`

La fusion du carbone demande encore un peu de travail. Il faut souvent rallentir la contraction qui précède, sinon le tracé dans le diagramme $\log T_c - \log \rho_c$ fait des sauts.

Pour les phases suivantes, il y a en principe peu à intervenir. Il faut régulièrement vérifier que le tracé dans le diagramme $\log T_c - \log \rho_c$ est bien lisse. Au besoin, il faut poser `ISLOW= 1`, voir `2` ou `3`.

Pendant la fusion de l'oxygène, il arrive un stade où le pas de temps devient si court que la convection ne peut plus être considérée comme instantanée. Si `IAUTO=1`, le programme change automatiquement `IDIFCON= 1`, sinon, il faut le poser à la main, par exemple lorsque le coeur convectif réapparaît.

7 Utilitaires

Plusieurs petits programmes utiles ont été développés ou adaptés pour `origin2010`. Ils peuvent être récupérés avec la commande :

```
svn checkout --username #username_mail
     https://subversion.unige.ch/users/ekstrom/Origin2013/tools/scripts
```

Les programmes suivants seront installés :

`makeini`: ce répertoire contient les routines d'un programme fortran permettant de créer les fichiers initiaux pour n'importe quelle étoile (`ini.rot`, `netdef.in`, `netalu.dat`). Après compilation (`make -f make_inifile`), l'appel se fait sans argument, le programme pose les questions au fur et à mesure.

`Origin_launch.py` : ce programme python permet d'enchaîner les séries de calcul pour l'évolution d'un modèle.
ATTENTION : avant de l'utiliser, il faut adapter l'adresse email ainsi que le chemin du programme d'évolution utilisé par défaut.
On le lance avec le petit nom de l'étoile. On peut lui spécifier un programme d'évolution différent que celui par défaut avec l'argument `-e #programme`. On peut lui spécifier de s'arrêter à un stade évolutif précis (`-p #phase`) ou à un numéro de modèle particulier (`-m #numéro`). Il envoie un mail à l'utilisateur en cas de plantage de l'étoile ou de fin de calcul, sauf si on l'a lancé avec l'option `-N`. Un fichier de résumé est généré, `computation.log`, qui mentionne le serveur de calcul, l'heure de début, le programme utilisé, les éventuels choix d'arrêt, et l'heure de fin du calcul.
Exemple d'utilisation : `python #chemin/Origin_launch.py P007z14S4 -p 4`
`python #chemin/Origin_launch.py P015z14S0 -m 5001 -N`
L'aide s'appelle en tapant la commande : `python #chemin/Origin_launch.py -h`.

`MakeInput` : ce script bash permet de recréer le fichier `.input` à partir d'un `.s` existant. L'appel se fait avec le nom des fichiers sans extension et le numéro du modèle. L'option -d permet d'aller chercher des fichiers ailleurs que dans le répertoire courant.
Exemple d'utilisation : `MakeInput P007z14S4 1021`
`MakeInput -d "../Be/P007z14S4" P007z14S4 1021`

`cleanfiles` : ce script bash nettoie le répertoire de calcul des fichiers inutiles, afin de réduire sa taille une fois le calcul terminé :

- élimination des fichiers vides et des petits fichiers inutiles ;
- concaténation des `.[a,g,s]0*` en `.wa`, `.wg` et `.ws` respectivement ;
- élimination des `.b` sauf ceux se terminant par `0` ou `.gz` ;
- élimination des `.l` sauf ceux se terminant par `001` ;
- zippage de tous les `.v` ;
- archivage des `.[a,g,z]` non-vides dans `[a,g,z]files.tgz` ;
- pour éviter les accidents, l'accès en écriture des fichiers `.w*` est interdit.

L'appel se fait avec le nom de l'étoile. En option, on peut calculer les zones de brûlage au cours de l'évolution en le lançant avec l'option `-b` (attention : nettement plus long !).
Exemple d'utilisation : `cleanfiles P007z14S4`
L'aide s'appelle en tapant la commande : `cleanfiles -h`.

`Origin_Tools` : c'est l'outil python de visualisation des modèles de Genève. Un mode d'emploi est fourni en annexe. Pour l'installer, il suffit de dézipper le fichier `OriginTools_latest.tar.gz`, d'aller dans le répertoire créé, et de taper la commande : `python setup.py install`.

8 Annexes

.g file (`Origin2013`)

n°	variable	format	grandeur	description
1	nm	i6		numéro du modèle
2	u1	1pe21.15		âge
3	u2	f10.6	$M/M_\odot$	
4	xl	f8.6	$\log(L/L_\odot)$	
5	xtt	f8.6	$\log(T_{\rm eff})$	
6	u5	e13.7	$X(1)$	)
7	u7	e13.7	$Y(1)$	)
8	b33	1pe13.7	^{3}He(1)	)
9	u8	1pe13.7	^{12}C(1)	)
10	u9	1pe13.7	^{13}C(1)	) abondances
11	u10	1pe13.7	^{14}N(1)	) en
12	u12	1pe13.7	^{16}O(1)	) surface
13	u13	1pe13.7	^{17}O(1)	)
14	u14	1pe13.7	^{18}O(1)	)
15	u6	1pe13.7	^{20}Ne(1)	)
16	u11	1pe13.7	^{22}Ne(1)	)
17	qmnc	f6.4	$M_{\rm cc}$	fraction de masse du coeur convectif
18	xte	f8.6	$\log(T_{\rm eff})_{\rm non\ corr.}$	$T_{\rm eff}$ non corrigée (*corrwind*)
19	xmdot	f7.3	$\log(\dot{M})$	
20	rhoc	f8.6	$\log(\rho_{\rm c})$	
21	tc	f8.6	$\log(T_{\rm c})$	
22	u15	e14.7	$X(\rm m)$	)
23	u17	e13.7	$Y(\rm m)$	)
24	b33m	1pe13.7	^{3}He(m)	)
25	u18	1pe13.7	^{12}C(m)	)
26	u19	1pe13.7	^{13}C(m)	)
27	u20	1pe13.7	^{14}N(m)	) abondances
28	u22	1pe13.7	^{16}O(m)	) au
29	u23	1pe13.7	^{17}O(m)	) centre
30	u24	1pe13.7	^{18}O(m)	)
31	u16	1pe13.7	^{20}Ne(m)	)
32	u21	1pe13.7	^{22}Ne(m)	)
33	ybe7m	1pe9.3	^{7}Be(m)	)
34	yb8m	1pe9.3	^{8}B(m)	)
35	flube7	1pe9.3		flux de ν dû à $^7{\rm Be}(e^-,\nu)^7{\rm Li}$
36	flub8	1pe9.3		flux de ν dû à $^8{\rm B}(e^+\nu)^8{\rm Be}$
37	snube7	1pe9.3		cf. 35 mais en SNU
38	snub8	1pe9.3		cf. 36 mais en SNU
39	rapcrj	f10.8	$\Omega/\Omega_{\rm crit}$	
40	rot1	1pe9.3	$\Omega(1)$	Ω en surface
41	rotm	1pe9.3	$\Omega(\rm m)$	Ω au centre
42	xobla	1pe9.3	$R_{\rm pol}/R_{\rm eq}$	
43	u27	1pe9.3	$^{26}{\rm Al}g(1)$	abondance en surface
44	u33	1pe9.3	$^{26}{\rm Al}g(\rm m)$	abondance au centre
45	alpro6	1pe9.3	$\dot{M}(\Omega)/\dot{M}(0)$	correction de la perte de masse dûe à la rotation
46	lcnom	i4		couche avec max de CNO
47	xmcno	f8.4		masse de la couche `lcnom`
48	scno	1pe8.2		somme des éléments CNO dans couche `lcnom`
49	xjspe1	f10.3	$j(3\ M_\odot)$	moment spécifique à 3 $M_\odot$[10^{16} cm^2 s^{-1}]
50	xjspe2	f10.3	$j(5\ M_\odot)$	moment spécifique à 5 $M_\odot$[10^{16} cm^2 s^{-1}]
51	vcri1m	1pe8.2	$v_{\rm crit,1}$	1$^{\rm re}$racine
52	vcri2m	1pe8.2	$v_{\rm crit,2}$	2$^{\rm e}$racine
53	vequam	1pe8.2	$v_{\rm eq}$	(avant convergence)
54	rapomm	f8.6	$\Omega/\Omega_{\rm crit}$	limite $\Gamma\Omega$
55	eddesm	f8.6	$\Gamma_{\rm Edd}$	
56	vcrit1	1pe8.2	$v_{\rm crit,1}$	1$^{\rm re}$racine
57	vcrit2	1pe8.2	$v_{\rm crit,2}$	2$^{\rm e}$racine
58	vequa	1pe8.2	$v_{\rm eq}$	(après convergence)
59	rapom2	f8.6	$\Omega/\Omega_{\rm crit}$	limite $\Gamma\Omega$
60	eddesc	f8.6	$\Gamma_{\rm Edd}$	
61	dmneed	1pe13.7	$\Delta M_{\rm need}$	masse devant être perdue à l'équateur (vitesse critique)
62	xmdotneed	1pe13.7	$\dot{M}_{\rm need}$	perte de masse nécessaire pour perdre $\Delta M_{\rm need}$
63	dlelex	1pe13.7	$\Delta\mathcal{L}_{\rm rad+aniso+méca}$	moment cinétique perdu sur un pas de temps (vents + mécanique) [10^{53} g cm^2 s^{-1}]
64	bmomit	1pe13.7	I	moment d'inertie total [10^{57} g cm^2]
65	btot	1pe13.7	$\mathcal{L}_{\rm tot}$	moment cinétique total [10^{53} g cm^2 s^{-1}]
66	ekrote	1pe13.7	$E_{\rm rot}$	énergie cinétique de la rotation [E_{51}]
67	epote	1pe13.7	$E_{\rm pot}$	énergie potentielle [E_{51}]
68	ekine	1pe13.7	$E_{\rm gaz}$	énergie thermique du gaz [E_{51}]
69	erade	1pe13.7	$E_{\rm rad}$	énergie de la radiation [E_{51}]
70 - 109	uudrawc	f5.3		limites des zones convectives.
110	btotatm	1pe16.10	$\mathcal{L}_{\rm tot}$	moment cinétique total (étoile + enveloppe)[10^{53} g cm^2 s^{-1}]

.v file (`Origin2013`)

nº	variable	format	grandeur	description
1	j	i4		nº de couche (1 : surface, m : centre)
2	vm	f10.7	M_r/M	
3	x2	f10.7	$\log(P_r)$	
4	x3	f10.7	$\log(T_r)$	
5	x4	f14.11	$\log(r)$	
6	vl	e14.6	$L/L_{\rm tot}$	
7	x	e14.7	X	
8	y	e14.7	Y	
9	xc12	e14.7	^{12}C	
10	xo16	e14.7	^{16}O	
11	eps	1pe11.4	ϵ(H)	énergie produite par H-b
12	epsy	1pe11.4	ϵ(He)	énergie produite par He-b
13	epsc	1pe11.4	ϵ(C)	énergie produite par C-b
14	radm	1pe11.4	$\nabla_{\rm rad}$	(moyenne entre `j` et `j+1`)
15	x6	f11.6	$\log(\rho)$	
16	zensi	1pe12.5		facteur de la convection (<0 : rad, >0 : conv.)
17	epsn	1pe11.4	ϵ_ν	énergie emportée par les ν
18	x10	1pe12.5	$\partial\ln\kappa/\partial\ln P$	
19	x11	1pe12.5	$\partial\ln\kappa/\partial\ln T$	
20	x12	1pe12.5	$\partial\ln E/\partial\ln P$	
21	x13	1pe12.5	$\partial\ln E/\partial\ln T$	
22	x14	1pe12.5	$\partial\ln\rho/\partial\ln P$	
23	x15	1pe12.5	$-\partial\ln\rho/\partial\ln T$	
24	psi	1pe9.2		degré de dégénérescence
25	epsyy	1pe9.2	$\epsilon(3\alpha)$	
26	epsyc	1pe10.2	$\epsilon(^{12}\mathrm{C}(\alpha,\gamma)^{16}\mathrm{O})$	
27	epsyo	1pe11.2	$\epsilon(^{16}\mathrm{O}(\alpha,\gamma)^{20}\mathrm{Ne})$	
28	eg	1pe12.5	$\epsilon_{\rm grav}$	
29	adim	1pe12.5	$\nabla_{\rm ad}$	(moyenne entre `j` et `j+1`)
30	x8	1pe12.5	κ(r)	opacité
31	beta	1pe12.5	$P_{\rm gaz}/P_{\rm tot}$	
32	y3	e14.7	^{3}He	
33	xc13	e14.7	^{13}C	
34	xn14	e14.7	^{14}N	
35	xn15	e14.7	^{15}N	
36	xo17	e14.7	^{17}O	
37	xo18	e14.7	^{18}O	
38	xne20	e14.7	^{20}Ne	
39	xne22	e14.7	^{22}Ne	
40	xmg24	e14.7	^{24}Mg	
41	xmg25	e14.7	^{25}Mg	
42	xmg26	e14.7	^{26}Mg	
43	vmyhelio	f9.6	μ	poids moléculaire moyen de la couche `j`
44	omegi	1pe12.5	$\Omega(r)$	vitesse angulaire
45	xdmudp	1pe12.5	∇_μ	gradient de composition chimique
46	xricha	1pe12.5	Ri	nombre de Richardson
47	difcon	1pe12.5	$D_{\rm conv}$	coefficient de diffusion convective
48	difsch	1pe12.5	$D_{\rm sh}$	coeff. de diffusion due au cisaillement
49	difdef	1pe12.5	$D_{\rm eff}$	coeff. de diffusion de la circ. mérid. et de la turb.
50	vmasse	f9.4	M_r	masse incluse dans la coquille [$\mathrm{M}_\odot$]
51	dlodlr	e15.8	$\partial\ln\Omega/\partial\ln r$	gradient de vitesse angulaire
52	xkther	e15.8	K	coeff. de diffusion thermique
53	ucicoe	e15.8	$U(r)$	composante verticale de la circ. mérid.
54	vcicoe	e15.8	$V(r)$	composante tangentielle de la circ. mérid.
55	dcirch	e15.8		coeff. de diffusion de circ. mérid. (seulement si `imagn=1`)
56	echpry	e15.8	H_P	échelle de pression
57	gravy	e15.8	g	gravité
58	dhori	e15.8	D_h	coeff. de diffusion de la turb. horiz.
59	omegp	e15.8	$\Omega(r)$	calculé seulement avec conserv. du mom. cin.
60	vr	e15.8	$r_{\rm prev.}$	rayon du modèle précédent
61	vomegi	e15.8	$\Omega(r)_{\rm prev.}$	vitesse ang. du modèle préc.
62	dmago	e15.8	$D_{\rm magn,\Omega}$	coeff. de diffusion magn. pour Ω
63	dmagx	e15.8	$D_{\rm magn}$	coeff. de diff. magn. pour les esp. chimiques
64	etask	e15.8	η	utilisé dans le traitement du magnétisme
65	bvais2	e15.8	N^2	carré de la fréquence de Brunt-Väisälä
66	bphi	e15.8	B_ϕ	champ magnétique toroïdal
67	alven	e15.8	ω_A	fréquence d'Alfven
68	xqmin	e15.8		grad. de rotation min. pour amplification du champ magn
69	vmye	f9.6	μ_e	poids moléculaire des e^-
70	xf19	1pe14.7	^{19}F	
71	xne21	1pe14.7	^{21}Ne	
72	xna23	1pe14.7	^{23}Na	
73	xal26g	1pe14.7	^{26}Alg	
74	xal27	1pe14.7	^{27}Al	
75	xsi28	1pe14.7	^{28}Si	
76	xc14	1pe14.7	^{14}C	
77	xf18	1pe14.7	^{18}F	
78	xneut	1pe14.7	n	
79	vxprot	1pe14.7	p	
80	xbid	1pe14.7		
81	abelx(1,8)	1pe14.7	^{28}Si	) seulement si `inetburn`$\geq$`1`
82	abelx(2,8)	1pe14.7	^{32}S	)
83	abelx(3,8)	1pe14.7	^{36}Ar	)
84	abelx(4,8)	1pe14.7	^{40}Ca	)
85	abelx(5,8)	1pe14.7	^{44}Ti	)
86	abelx(6,8)	1pe14.7	^{48}Cr	)
87	abelx(7,8)	1pe14.7	^{52}Fe	)
88	abelx(8,8)	1pe14.7	^{56}Ni	)
89	btotq	1pe14.7	$\mathcal{L}_{\rm couche}$	moment cinétique de la couche
90	xomegafit	1pe14.7	fit de Ω	
91	xmufit	1pe14.7	fit de μ	
92	1/amu	1pe14.7	μ utilisé pour le fit	
93	xoblaj	f8.6	$r_{\rm pol}/r_{\rm eq}$ dans la couche	

Sylvia Ekström October 28, 2014

Quick start manual for ORIGIN_TOOLS
(v. 4.0.3)

1 Reading of files

- **Loading an EVOL file**
  Command: `loadE('#file',num_⋆,num_start=#line,num_end=#line,format='format')`
  Example: `loadE('~/calc/Z002/P015z02S0/P015z02S0.wg',1)`
  `loadE('~/grids/tables/M015z02S0.dat',2,format='tgrids')`
  The existing formats are `o2013,tgrids,tools` (default: `o2013`).

- **Loading a STRUC file**
  Command: `loadS('#file',num_⋆,num_start=#line,format='format')`
  Example: `loadS('P015z02S0.v0001001',1`
  `loadS('P015z02S0_StrucData_0001001.dat',2,format='full')`
  The existing formats are `o2013,full` (default: `o2013`).

NB: switching from EVOL mode to STRUC mode with `switch('struc')` and back with `switch('evol')`

2 Models selection

- **Selection:**
  Command: `select_model(num_⋆)`
  Example: `select_model(2)`
  `select_model([1,2,3,4,5])`

- **Addition of a model:**
  Command: `add_model(num_⋆)`

- **Removal of a model:**
  Command: `del_model(num_⋆)`

- **Recall of loaded models:**
  Command: `LoadedEvol()` (EVOL)
  Command: `LoadedStruc()` (STRUC)

3 Variables

- **List of available variables**
  Command: `VarEvol()` (EVOL)
  Command: `VarStruc()` (STRUC)
  (see also section 6)

- **Get the values of a variable for a model**
  Command: `Get_Var('variable',num_⋆)`

- **Addition of a new variable**
  Command: `Set_Var(newVar,'Var_Name',num_⋆,label='#LaTeX_string',category='category')`
  with the following existing categories (but it's possible to create one) :
  - EVOL : `model,surface,centre,abundances,rotation,winds,energetics`
  - STRUC : `structure,thermo,EOS,energy,abundances,rotation,magnetism`

  Example: `VVc = Get_Var('Vsurf',1)/Get_Var('Vcrit1',1)`
  `Set_Var(VVc,'VVc',1,label='$V/V_\mathrm{crit}$',category='rotation')`

4 Graphics

4.1 Basics

- **Definition of the x axis**
  Command: `defX('variable')`

As default, the axis is t [Myr] in EVOL mode and M_r in STRUCmode

- **Basic command**
  Command: `Plot('variable')`

- **Graph with variable(s) in log**
  Command: `log_Var('axis')`
  with `axis=x,y,xy`, then `Plot('variable')`

- **Ratio between two variables**
  Command: `plotRatio('var_1','var_2')`
  The optional argument `index=#line` allows to divide variable 1 by the value of variable 2 at that line. Usefull for example to plot the relative evolution of a variable with respect to its initial value :
  `plotRatio('var','var',index=0)`.

- **3rd variable in colour code over the curve**
  Command: `Plot_colour('var_y','var_z')`
  The min and max values of `var_z` are determined automatically, but it is possible to modify this behaviour with `CBLimits(min=#min,max=#max)`, each argument being optional. To get back to an automatic determination, the command is `no_CBLimits()`, which accepts the argument `min` or `max` if only one value has to be redetermined.
  The colour map can be changed with the command `set_colourMap('map')` where `'map'` is one of the colour maps proposed by `matplotlib` (see the documentation page)

4.2 Predefined graphs

EVOL:

- **HR diagram**
  Command: `HRD_plot()`

- **Kippenhahn diagram**
  Command: `Kippen(num_⋆)`

- **log T_c − log ρ_c diagram**
  Command: `rhoT()`

- **Y − log T_{eff} diagram**
  Command: `YTeff()`

- **Abundances evolution**
  Command: `Abund('c')` (centre)
  Command: `Abund('s')` (surface)

- **N/C vs N/O diagram**
  Command: `NCNO()`

- **Set of plots summarising a model**
  Command: `Hirschi_plots(num_⋆)`
  This command automatically generates the following plots:
  - window 1: HRD, Abund(c), Kippenhahn, L and T_{eff} vs time
  - window 2 : V_{surf}, Ω/Ω_{crit}, $\dot{M}$, Ω_c/Ω_s
  - window 3 : Abund(s), N/O, N/H, N/C
  - window 4 : $T_c - \rho_c$, T_c and ρ_c vs time

  With the optional argument `'legend'`, a legend is added to the plots.

STRUC:

- **log T − log ρ diagram**
  Command: `rhoT()`

- **Gradients (∇_{ad} - ∇_{rad} - ∇_{μ})**
  Command: `Nablas(num_⋆)`

- **Abundances profile**
  Command: `Abund('p')`

- **addition of the convective zones on the plot**
  Command: `convZones(num_⋆)`

- **Set of plots summarising a structure**
  Command: `Hirschi_plots(num_⋆)`
  This command automatically generates the following plots:
  - window 1 : Abund(p)
  - window 2 : T, r, P, ρ
  - window 3 : L and ϵ, T vs ρ, ϵ_{reac} and ϵ_{grav}, nablas and kappa
  - window 4 : D, N^2, c_{sound} eandt V_{MLT}, magn. field variables
  - window 5 : Ω and $\Omega/\Omega_{\text{crit}}$, V_{eq}, j, U and V

  With the optional argument `'legend'`, a legend is added to the plots.

4.3 Saving the figures

The python window allows to save the current figure in png format, with a clic on .

For a vectorised format, the command `MyFig('#file_name')` can be used. The default format is pdf, but it might be customised with the optional argument `format='#format'`.

5 Adaptations to specific needs

5.1 Axes

- **Change of the limits**
  Command: `Limits(xmin=,xmax=,ymin=,ymax=)`
  NB: back to normal with `noLimits('x','y' ou 'xy')`. `noLimits()` without argument corresponds to `'xy'`.

- **Inversion of an axis**
  Command: `axis_inv('x','y' or 'xy')`
  NB: back to normal with `no_axis_inv('x','y' ou 'xy')`. `no_axis_inv()` without argument corresponds to `'xy'`.

- **Axis in logarithmic scale**
  Command: `logScale('x','y' or 'xy')`
  NB: back to normal with `no_logScale()`

- **Modification of an axis label**
  Command: `change_label('axis','label')`

5.2 Points/curves

- **Switch from curves to points**
  Command: `Points(True)`

- **Choice of a colour sequence when several models are plotted together**
  Command: `set_colourSequence('i' or 'c')`
  `'c'` stands for `'contrast'`: black,red,green,blue,cyan,magenta,orange,olive,pink,brown,grey
  `'i'` stands for `'iris'`: black,blue,cyan,green,yellow,orange,red,magenta,purple,grey

- **Choice of line style**
  Command: `setLine_style('style')`
  where `'style'` can be:
  - `cycle_colour` so that the line style changes only after a whole colour sequence;
  - `cycle_all` so that each curve is drawn with a different style;
  - any of the following: `'-', '- -', ':', '-.'`

- **Choice of point style**
  Command: `setPoint_style('style')`
  where `'style'` can be:
  - `cycle_colour` so that the points style changes only after a whole colour sequence;
  - `cycle_all` so that each model points are drawn with a different style;
  - any of the following: `'o','^','*','s','p','v','d','>','<'`

5.3 Windows

- **Drawing of 2 or 4 plots on the same window**
  Command: `multiPlot(2 or 4)`
  NB: back to normal with `multiPlot(1)`

5.4 Navigation within the graphs

The python window allows to easily zoom on a part of the graph after a clic on . It is also possible to navigate within the graph after clicking on . The original view is recovered with the button.

- **Activation of the cursor**
  Command: `cursor()`
  The coordinates of clicks are written on the terminal; to exit the cursor mode, just hit `Enter` on the terminal window.

- **Measurement of the distance between two points**
  Command: `dist()`
  The clics are validated by hitting `Enter` on the terminal window. The coordinates are written there, as well as Δx, Δy and the distance.

5.5 Interventions on the graphs

- **Superimposing plots**
  Command: `plot2var(mode)`
  By default, the mode is `'same'` that draws all the variables on the same y axis. The mode mode `'double'` allows to draw the 2nd variable with an independent axis on the right.
  NB: back to normal with `plot1var()`

- **Drawing a vertical line at the x value or a horizontal line at the y value**
  Command: `xline(x_value), yline(y_value)`

- **Drawing a line from(x_1,y_1) to (x_2,y_2)**
  Command: `line(`x_1`,`x_2`,`y_1`,`y_2`)`

- **Drawing a dot at the coordinate (x,y)**
  Command: `dotxy(x,y)`

- **Addition of external data on a graph**
  Command: `plotExternal('#File_name', x_column,y_column)`
  The following optional arguments are accepted:
  - `skip=...` to skip header lines in the file `#File_name` ;
  - `style=...` to define the points/lines style (example: `'ro','b+','g.'`) ;
  - `log='x','y' or 'xy'` to take the log of the variable(s) ;
  - `new` to create a new figure rather than adding the data to the current figure.

- **Addition of text in the graph**
  Command: `x,y,'string'`
  The optional argument `fontsize=...` modifies the fontsize (default: 24)

- **Addition of a title at the top of the window**
  Command: `top_label('string')`
  The optional argument `fontsize=...` modifies the fontsize (default: 24)

6 List of the variables

6.1 In EVOL mode

- MODEL:

line :	model num	t_tauH :	$t/\tau_{\rm H}$
M :	M $[M_\odot]$	ageadv :	$\log$(time before collapse [yr])
t :	t [yr]	tauKH :	τ_{KH} [yr]
t6 :	t [Myr]	rhom :	$\rho_{\rm m}$ [g cm^3]
t9 :	t [Gyr]		

Besides, the following global variables are known:

FileName :	`#loaded_file`
Mini :	$M_{\rm ini}$ $[M_\odot]$
tau :	lifetimes [yr] ([$\tau_{\rm H}$,$\tau_{\rm He}$,$\tau_{\rm C}$,$\tau_{\rm Ne}$,$\tau_{\rm O}$])
format :	`#format`

- CENTRE:

Mcc :	$M_{\rm cc}$ $[M_\odot]$
Mccrel :	$M_{\rm cc}/M_{\rm tot}$
rhoc :	$\log(\rho_{\rm c}$ [g cm^{-3}])
Tc :	$\log(T_{\rm c}$ $[K]$)

- SURFACE:

Teff :	$\log(T_{\rm eff}$ [K])	R :	R $[R_\odot]$
Teffncorr :	$\log(T_{\rm eff}$ [K]) not corrected for the wind thickness (WR)	Rpol :	$R_{\rm pol}$ $[R_\odot]$
L :	$\log(L/L_\odot)$	gsurf :	$\log(g_{\rm surf}$ [cm s^{-2}])
Mbol :	$M_{\rm bol}$	gpol :	$\log(g_{\rm pol}$ [cm s^{-2}])
GammaEdd :	$\Gamma_{\rm Edd}$	ZCext :	$M_{\rm ZC,ext}$

- ROTATION:

Omega_surf :	$\Omega_{\rm surf}$ [s^{-1}]	GammaOmega :	$\Omega/min(\Omega_{\rm crit,1},\Omega_{\rm crit,2})$
Omega_cen :	$\Omega_{\rm cen}$ [s^{-1}]	rot_corr :	F_Ω
OOc :	$\Omega/\Omega_{\rm crit}$	Ltot :	$\mathcal{L}_{\rm tot}$ [10^{53} g cm^2 s^{-1}]
Vsurf :	$V_{\rm surf}$ [km s^{-1}]	Ltotint :	$\mathcal{L}_{\rm tot,int}$ [10^{53} g cm^2 s^{-1}]
Vcrit1 :	$V_{\rm crit,1}$ [km s^{-1}]	jspe5 :	$j_{5M_\odot}$ [10^{16} cm^2 s^{-1}]
Vcrit2 :	$V_{\rm crit,2}$ [km s^{-1}]	jspe3 :	$j_{3M_\odot}$ [10^{16} cm^2 s^{-1}]
oblat :	$R_{\rm pol}/R_{\rm eq}$	mominert :	I [10^{57} g cm^2]

- WINDS:

Mdot :	$\log(\dot{M}$ [$M_\odot$ yr^{-1}])	Vesc :	$V_{\rm esc}$ [km s^{-1}]
Mdot_mech :	$(\log(\dot{M})_{\rm mech}$ [$M_\odot$ yr^{-1}])	dlelex :	$\Delta\,\mathcal{L}_{\rm rad+aniso+mech}$ [10^{53} g cm^2 s^{-1}]
dMmech :	d$M_{\rm mech}$ [$M_\odot$]	Bmin :	$B_{\rm min}$ [G] (minimal magn. field for a wind-surface coupling)
Pwinds :	$P_{\rm winds}$ [erg s^{-1}]		
Vinf :	V_∞ [km s^{-1}]		

- ENERGETICS:

Epot :	$E_{\rm pot}$ $[E_{51}]$
Egaz :	$E_{\rm th,gaz}$ $[E_{51}]$
Erad :	$E_{\rm rad}$ $[E_{51}]$
Erot :	$E_{\rm rot}$ $[E_{51}]$

- ABUNDANCES:

H1s,H1c :	^{1}H (surf., centr.) [mass frac.]	Be7c :	^{7}Be (centr.) [mass frac.]
He3s,He3c :	^{3}He (surf., centr.) [mass frac.]	B8c :	^{8}B (centr.) [mass frac.]
He4s,He4c :	^{4}He (surf., centr.) [mass frac.]	C12s,C12c :	^{12}C (surf., centr.) [mass frac.]

C13s,C13c :	^{13}C (surf., centr.) [mass frac.]	C12C13 :	$\log(^{12}\mathrm{C}/^{13}\mathrm{C}$ [numb.])
N14s,N14c :	^{14}N (surf., centr.) [mass frac.]	C12C13rel :	$\log(^{12}\mathrm{C}/^{13}\mathrm{C}) - \log(^{12}\mathrm{C}/^{13}\mathrm{C})_{\mathrm{ini}}$
O16s,O16c :	^{16}O (surf., centr.) [mass frac.]	NH :	$\log(\mathrm{N/H}$ [numb.] $+ 12)$
O17s,O17c :	^{17}O (surf., centr.) [mass frac.]	NHrel :	$\log(\mathrm{N/H}) - \log(\mathrm{N/H})_{\mathrm{ini}}$
O18s,O18c :	^{18}O (surf., centr.) [mass frac.]	NC :	$\log(\mathrm{N/C}$ [numb.])
Ne20s,Ne20c :	^{20}Ne (surf., centr.) [mass frac.]	NCrel :	$\log(\mathrm{N/C}) - \log(\mathrm{N/C})_{\mathrm{ini}}$
Ne22s,Ne22c :	^{22}Ne (surf., centr.) [mass frac.]	NO :	$\log(\mathrm{N/O}$ [numb.])
Al26s,Al26c :	^{26}Al (surf., centr.) [mass frac.]	NOrel :	$\log(\mathrm{N/O}) - \log(\mathrm{N/O})_{\mathrm{ini}}$
Zsurf :	Z_{surf} [mass frac.]		

6.2 In STRUC mode

- STRUCTURE:

shell :	shell number	rprev :	r_{prev} $[R_\odot]$
Mfrac :	M_r/M_{tot}	g :	g_r [cm s^{-2}]
Mr :	M_r $[M_\odot]$	N2 :	N^2 [s^{-1}]
r_cm :	r [cm]	Nmu2 :	N^2_μ [s^{-1}]
r :	r $[R_\odot]$	NT2 :	N^2_T [s^{-1}]

Besides, the following global variables are known:

M_tot :	M_{tot} $[M_\odot]$
FileName :	`#loaded_file`
Model :	model number
nshell :	total shells number
age :	age [yr]
timestep :	$\delta\, t$ [s]

- ENERGY:

L :	L_r/L_{tot}	epsOagNe :	$\epsilon(^{16}O(\alpha,\gamma)^{20}Ne)$ [erg g^{-1}s^{-1}]
epsH :	ϵ(H) [erg g^{-1}s^{-1}]	epsnu :	$-\epsilon_\nu$ [erg g^{-1}s^{-1}]
epsHe :	ϵ(He) [erg g^{-1}s^{-1}]	eps_reac :	$\epsilon_{\mathrm{nucl}} + \epsilon_\nu$ [erg g^{-1}s^{-1}]
epsC :	ϵ(C) [erg g^{-1}s^{-1}]	epsgrav :	ϵ_{grav} [erg g^{-1}s^{-1}]
eps3a :	$\epsilon(3\alpha)$ [erg g^{-1}s^{-1}]	dEdP :	$\mathrm{d}\ln E/\mathrm{d}\ln P$
epsCagO :	$\epsilon(^{12}C(\alpha,\gamma)^{16}O)$ [erg g^{-1}s^{-1}]	dEdT :	$\mathrm{d}\ln E/\mathrm{d}\ln T$

- EOS:

rho :	ρ [g cm^3]	Nabmu :	∇_μ
mu :	μ	drhodP :	$\mathrm{d}\ln\rho/\mathrm{d}\ln P$
muprev :	μ_{prev}	delta :	$\delta = -\mathrm{d}\ln\rho/\mathrm{d}\ln T$
mufit :	μ_{fit}	cs :	c_{sound} [cm s^{-1}]
mue :	μ_e	psi :	ψ (indicator of degeneracy)

- THERMO:

P :	P [g cm^{-1} s^{-2}]	Nabrad :	∇_{rad}
beta :	$\beta = P_{\mathrm{gas}}/P_{\mathrm{tot}}$	Nabla_int :	∇_{int}
Hp :	H_P [cm]	kappa :	κ [cm^2 g^{-1}]
T :	T $[K]$	dkdP :	$\mathrm{d}\ln\kappa/\mathrm{d}\ln P$
Kther :	K_{ther} [cm^2 s^{-1}]	dkdT :	$\mathrm{d}\ln\kappa/\mathrm{d}\ln T$
Nabla :	∇	Cp :	C_P [ergs g^{-1} K^{-1}]
Nabad :	∇_{ad}	V_MLT :	V_{MLT} [cm s^{-1}]

- ROTATION:

Omega :	Ω [s^{-1}]	Vr :	V_r [cm s^{-1}]
Omegaprev :	Ω_{prev} [s^{-1}]	Veq :	V_{eq} [km s^{-1}]
Omegacons :	$\Omega_{\mathcal{L},\mathrm{cons.}}$ [s^{-1}]	Lang :	$\mathcal{L}_r$ [g cm^2 s^{-1}]
Omfit :	Ω_{fit} [s^{-1}]	jr :	j_r [cm^{2}s^{-1}]
dlodlr :	$\mathrm{d}\ln\Omega/\mathrm{d}\ln r$	jS :	$j_{\mathrm{Schwarzschild}}$ [cm^{2}s^{-1}]
obla :	$r_{\mathrm{pol}}/r_{\mathrm{eq}}$	jK :	j_{Kerr} [cm^{2}s^{-1}]

jKmax :	$j_{\rm Kerr}^{\rm max}$ [cm^{2}s^{-1}]	Dconv :	$D_{\rm conv}$ [cm^2 s^{-1}]
Dh :	$D_{\rm h}$ [cm^2 s^{-1}]	Ur :	U_r [cm s^{-1}]
Dshear :	$D_{\rm shear}$ [cm^2 s^{-1}]	Richardson :	Ri $= N^2/({\rm d}V/{\rm d}z)^2$
Deff :	$D_{\rm eff}$ [cm^2 s^{-1}]		
Dcirc :	$D_{\rm circ}$ [cm^2 s^{-1}]		

• MAGNETISM:

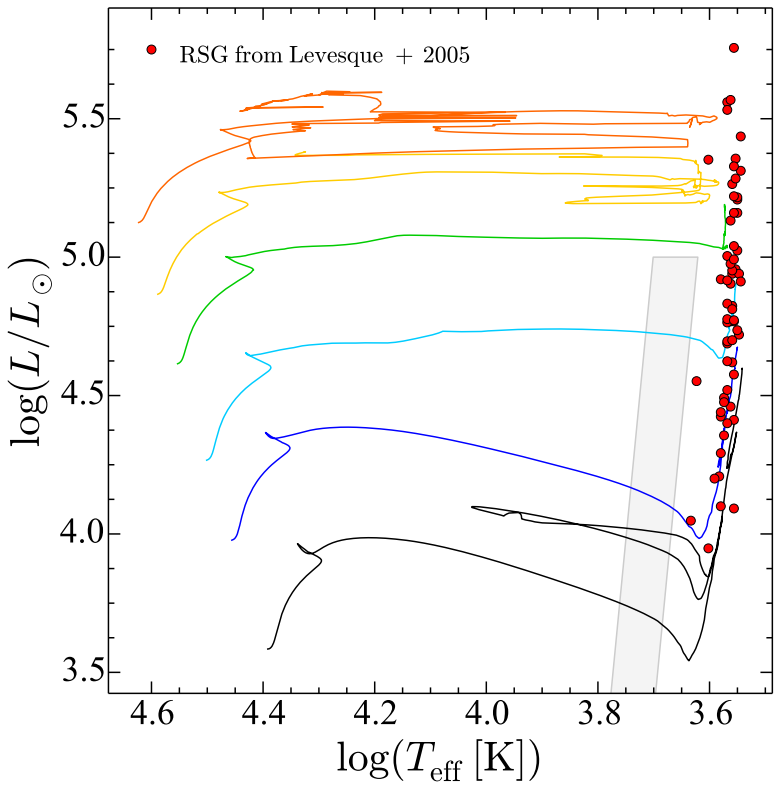
Br :	B_r $[G]$	etask :	η/K
Bphi :	B_ϕ $[G]$	N2mag :	$N^2_{\rm mag}$ [s^{-1}]
qmin :	$q_{\rm min}$	DmagO :	$D_{{\rm mag},\Omega}$ [cm^2 s^{-1}]
alfven :	$\omega_{\rm Alfven}$ [s^{-1}]	DmagX :	$D_{{\rm mag},X}$ [cm^2 s^{-1}]

• ABUNDANCES:

H1 :	^{1}H [mass frac.]	Si28,Si28_alu :	^{28}Si [mass frac.]
He3,He4 :	^{3,4}He [mass frac.]	S32 :	^{32}S [mass frac.]
C12,C13,C14 :	^{12,13,14}C [mass frac.]	Ar36 :	^{36}Ar [mass frac.]
N14,N15 :	^{14,15}N [mass frac.]	Ca40 :	^{40}Ca [mass frac.]
O16,O17,O18 :	^{16,17,18}O [mass frac.]	Ti44 :	^{44}Ti [mass frac.]
F18,F19 :	^{18,19}F [mass frac.]	Cr48 :	^{48}Cr [mass frac.]
Ne20,Ne21,Ne22 :	^{20,21,22}Ne [mass frac.]	Fe52 :	^{52}Fe [mass frac.]
Na23 :	^{23}Na [mass frac.]	Ni56 :	^{56}Ni [mass frac.]
Mg24,Mg25,Mg26 :	^{24,25,26}Mg [mass frac.]	protons :	protons [mass frac.]
Al26,Al27 :	^{26,27}Al [mass frac.]	neutrons :	neutrons [mass frac.]

7 Examples

7.1 HR diagram with observational data points



Loading of the package:

```
from origin_tools.Origin_Tools import *
```

Loading of the evolution files:

```
loadE('~/OBS/Grids2010/wgred/Z014/P009z14S0.dat',1)
loadE('~/OBS/Grids2010/wgred/Z014/P012z14S0.dat',2)
loadE('~/OBS/Grids2010/wgred/Z014/P015z14S0.dat',3)
loadE('~/OBS/Grids2010/wgred/Z014/P020z14S0.dat',4)
loadE('~/OBS/Grids2010/wgred/Z014/P025z14S0.dat',5)
loadE('~/OBS/Grids2010/tables/Z014/M032Z14V0.dat',6,format='tgrids')
```

Choice of the colour sequence `'iris'`:

```
set_colourSequence('i')
```

Change of the upper limit in y:

```
Limits(ymax=5.9)
```

Drawing of the HR diagram (predefined command):

```
HRD_plot()
```

Passing to `'points'` mode:

```
Points(True)
```

Drawing of external data, read in a file:

```
plotExternal('RSG_Levesque2005.dat',0,2,log='x',style='ro')
```

Drawing of a dot for the legend:

```
dotxy(4.6,5.75,style='ro')
```

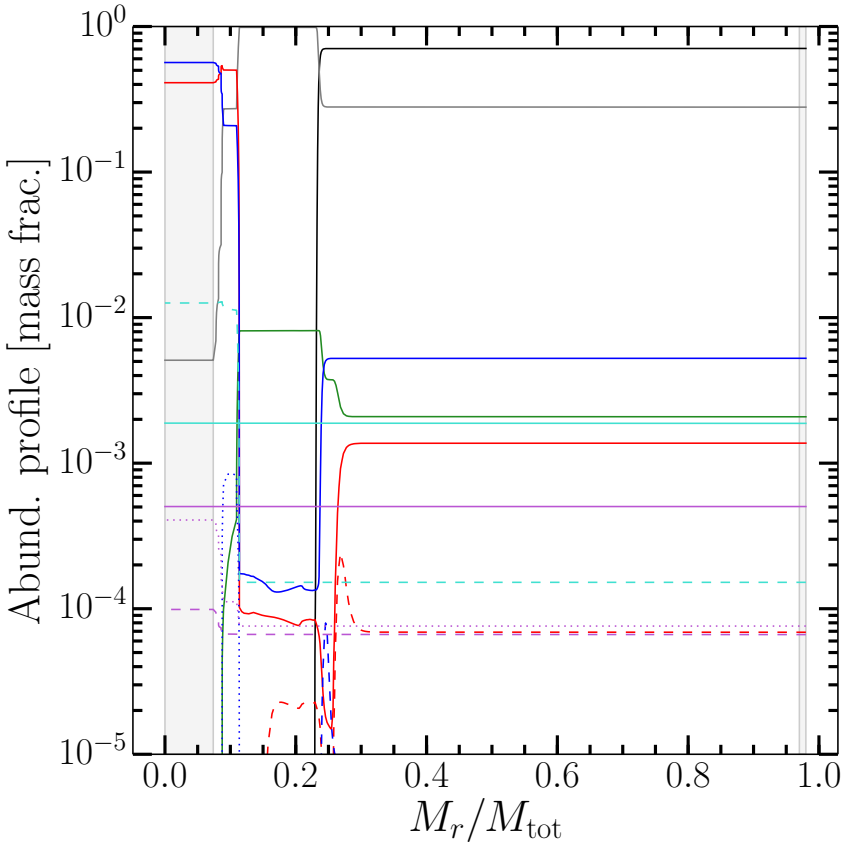
Addition of the legend text:

```
add_label(4.55,5.7,'$\mathrm{RSG\ from\ Levesque+2005}$',fontsize=16)
```

Saving the figure:

```
MyFig('HRDrainbow_RSG')
```

7.2 Abundances profiles with the convective zones shaded



Loading of a structure file:

```
loadS('~/OBS/Calc/testVrac/P007z14S0.v0022721',1)
```

Labels written with LaTeX:

```
iLatex(True)
```

Enhancement of the axes ticks:

```
ChangeTickSize(length=12,width=2)
```

Modification of the lower limit in y:

```
Limits(ymin=1.e-5)
```

y-axis in logarithmic scale:

```
logScale('y')
```

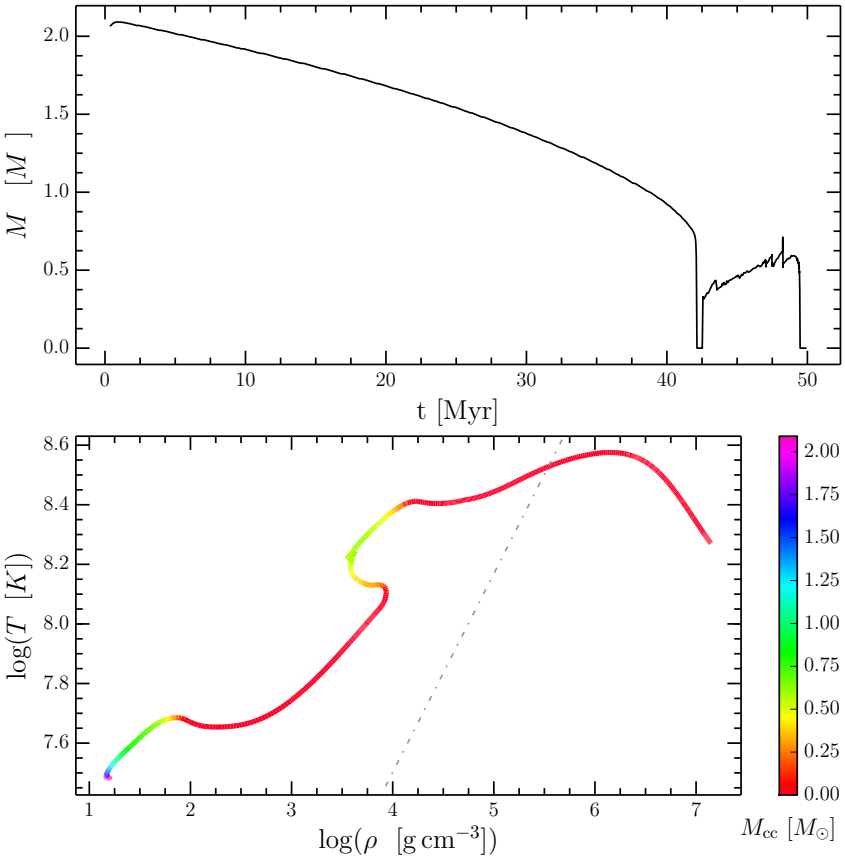
Drawing of the abundances profiles (predefined command):

```
Abund('p')
```

Addition of the convective zones as shaded areas:

```
convZones(1)
```

7.3 Double figure with one of them having a 3rd variable in colour code



Loading of an evolution file:

```
loadE('/Users/ekstrom/OBS/Grids2010/wgred/Z014/P007z14S0.dat',1)
```

Labels written with LaTeX:

```
iLatex(True)
```

Passing to two figures in a window mode:

```
multiPlot(2)
```

Reduction of the labels size:

```
ChangeFontSize(12)
```

Drawing of the firrst figure (default x-axis):

```
Plot('Mcc')
```

Choice of the x-axis for the 2nd figure:

```
defX('rhoc')
```

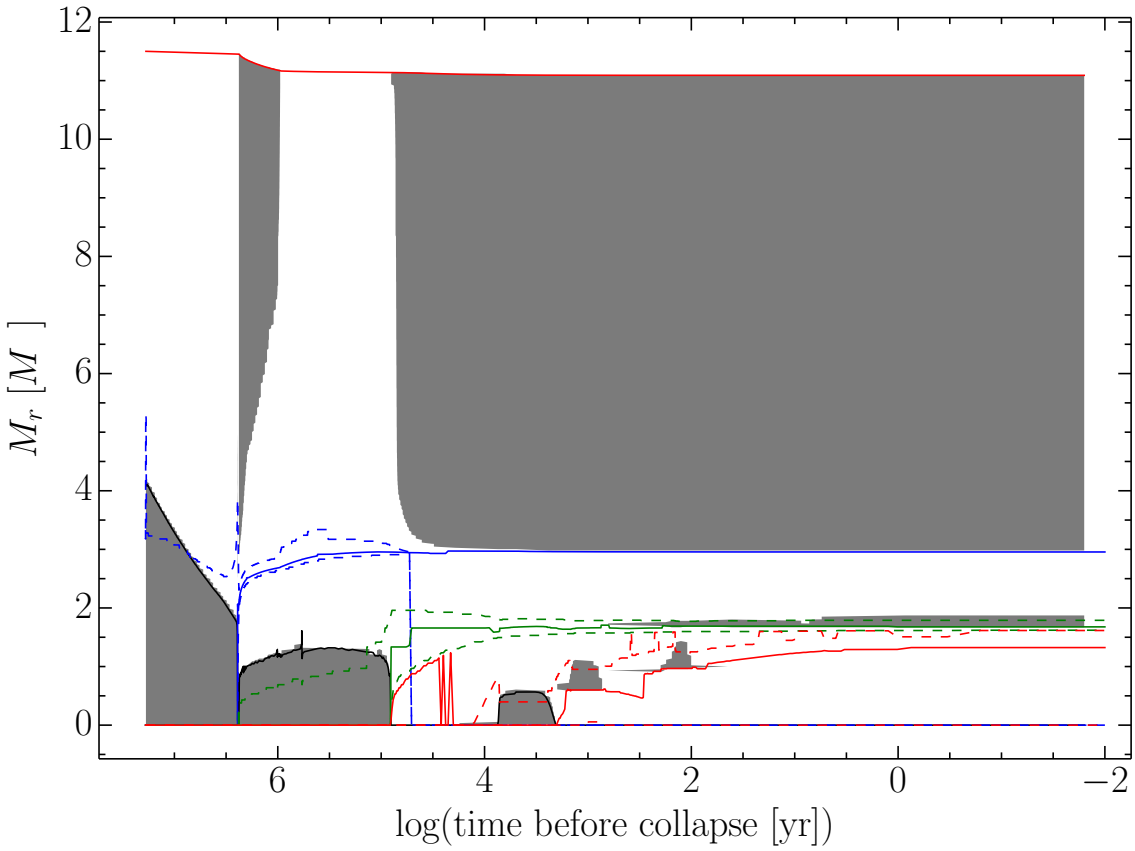
Drawing with a 3rd variable (`Mcc`) in colour code:

```
Plot_colour('Tc','Mcc')
```

Addition of the gas degeneracy line on the plot:

```
degenerate_line()
```

7.4 Kippenhahn diagram with burning zones



Loading of an evolution file:

```
loadE('/Users/ekstrom/OBS/Grids2010/wgred/Z014/P011p5z14S0.dat',1)
```

Labels written with LaTeX:

```
iLatex(True)
```

Definition of the x variable:

```
defX('ageadv')
```

Drawing of the Kippenhahn diagram (predefined command):

```
Kippen(1,burn=True)
```

NB : the burning zones (plotted with the optional argument `burn=True`) are read in a `#starname.burn` file that must exist somewhere. This `.burn` file is automatically generated when the star's directory has been cleaned with the `cleanfiles` routine with the option `-b`. it can be generated afterwards on the `.v` file of a model with the script `BurningZonesCalc.bash` that runs on the `strucData` program. All those programs and routines can be found on the `svn`.